

Data Analytics (DA)

Module-End -5

Module End Assignment: Python For Data Analysis

“Social Media Engagement Analytics”

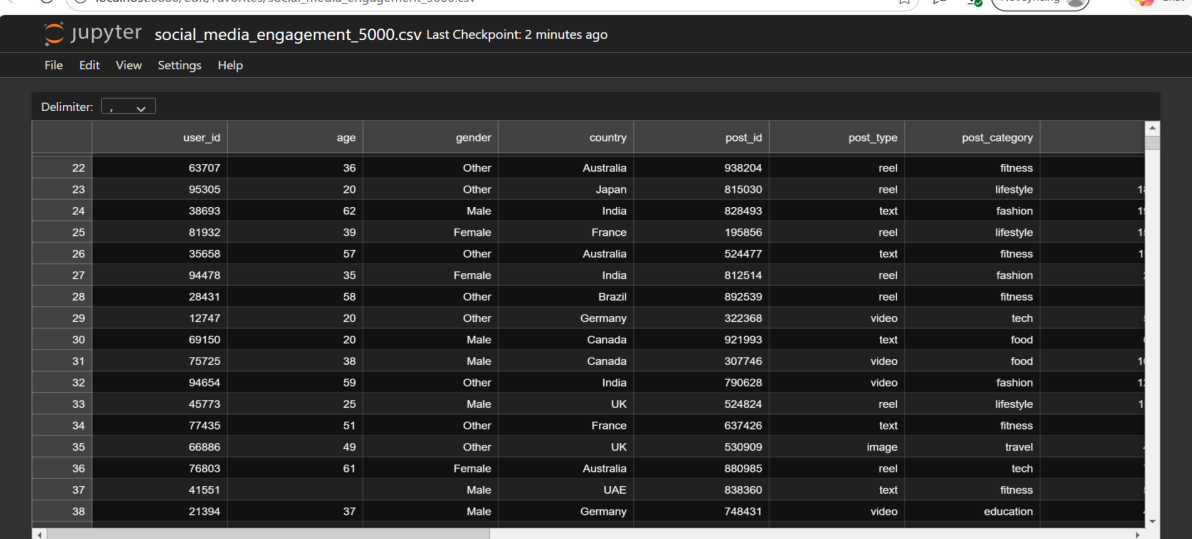
Project Title: Social Media Engagement Analytics Using Python

Problem Statement

Social media platforms generate massive volumes of engagement data—likes, comments, shares, impressions, watch time, and more. Analyzing such data helps companies understand user behavior, identify trends, and improve content performance.

In this assignment, you will work with a social media dataset containing social media engagement metrics. The task involves data cleaning, transformation, NumPy/Pandas operations, exploratory data analysis, visualizations, and generating insights.

Dataset Provided: {social_media_engagement_5000.csv }



social_media_engagement_5000.csv Last Checkpoint: 2 minutes ago

	user_id	age	gender	country	post_id	post_type	post_category
22	63707	36	Other	Australia	938204	reel	fitness
23	95305	20	Other	Japan	815030	reel	lifestyle
24	38693	62	Male	India	828493	text	fashion
25	81932	39	Female	France	195856	reel	lifestyle
26	35668	57	Other	Australia	524477	text	fitness
27	94478	35	Female	India	812514	reel	fashion
28	28431	58	Other	Brazil	892539	reel	fitness
29	12747	20	Other	Germany	322368	video	tech
30	69150	20	Male	Canada	921993	text	food
31	75725	38	Male	Canada	307746	video	food
32	94654	59	Other	India	790628	video	fashion
33	45773	25	Male	UK	524824	reel	lifestyle
34	77435	51	Other	France	637426	text	fitness
35	66886	49	Other	UK	530909	image	travel
36	76803	61	Female	Australia	880985	reel	tech
37	41551		Male	UAE	838360	text	fitness
38	21394	37	Male	Germany	748431	video	education

Assignment Tasks:

❖ Task 1 — Data Import & Setup

- Import CSV using Pandas
- Check/convert data types
- Convert date columns to datetime

Jupyter Python Code World MOD 5 Social media Last Checkpoint: 32 minutes ago

File Edit View Run Kernel Settings Help

Python 3 (ipykernel)

Python Code World MOD 5 Social media Last Checkpoint: 32 minutes ago

Python 3 (ipykernel)

Social Media Engagement Analytics

```
[1]: # Task 1 - Data Import & Setup

[7]: import pandas as pd
import numpy as np

# Load dataset
df = pd.read_csv("social_media_engagement_5000.csv")

# Display first 5 rows
display(df.head())

# Check dataset information
df.info()

# Check shape
print("Dataset Shape:", df.shape)

# Check column names
print("Column Names:")
print(df.columns.tolist())

# Check data types
print("\nData Types:")
print(df.dtypes)
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 19 columns):
 #   Column                Non-Null Count  Dtype  
---  --
 0   user_id               5000 non-null   int64   
 1   age                   4850 non-null   float64  
 2   gender                4850 non-null   str      
 3   country               5000 non-null   str      
 4   post_id               5000 non-null   int64   
 5   post_type             5000 non-null   str      
 6   post_category         5000 non-null   str      
 7   likes                 4850 non-null   float64  
 8   comments              4850 non-null   float64  
 9   shares                4850 non-null   float64  
10  watch_time_sec        5000 non-null   int64   
11  impression_count      5000 non-null   int64   
12  posted_at             5000 non-null   str      
13  follower_count        5000 non-null   int64   
14  is_verified           5000 non-null   bool     
15  device_type           5000 non-null   str      
16  sentiment              4850 non-null   str      
17  hashtags              5000 non-null   str      
18  engagement_rate        5000 non-null   float64  
dtypes: bool(1), float64(5), int64(5), str(8)
memory usage: 996.8 KB
Dataset Shape: (5000, 19)
Column Names:
```

```
Data Types:
user_id      int64
age          float64
gender       str
country      str
post_id      int64
post_type    str
post_category str
likes        float64
comments     float64
shares       float64
watch_time_sec int64
impression_count int64
posted_at    str
follower_count int64
is_verified  bool
device_type  str
sentiment    str
hashtags     str
engagement_rate float64
dtype: object
```

```
device_type    str
sentiment      str
hashtags       str
engagement_rate float64
dtype: object

Missing Values:
user_id      0
age          150
gender       150
country      0
post_id      0
post_type    0
post_category 0
likes        150
comments     150
shares       150
watch_time_sec 0
impression_count 0
posted_at    0
follower_count 0
is_verified  0
device_type  0
sentiment    150
hashtags     0
engagement_rate 0
dtype: int64

Duplicate Rows: 0
```

❖ Task 2 — Data Cleaning

• Cleaning Missing Data

- Detect missing values (isnull(), isna())
- Handle using: dropna(), fillna(), median/mode, forward/backward-fill

Python Code:

```
localhost:8888/notebooks/Favorites%2FPython%20Code%20World%20MOD%205%20Social%20media.ipynb
Jupyter Python Code World MOD 5 Social media Last Checkpoint: 36 minutes ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)
dtype: int64
Duplicate Rows: 0
[8]: # Task 2 - Data Cleaning: Missing Data
[9]: # Check missing values
print("Missing values before cleaning:")
print(df.isnull().sum())

# Separate numerical and categorical columns
numeric_columns = df.select_dtypes(include=np.number).columns
categorical_columns = df.select_dtypes(include="object").columns

# Fill numerical missing values with median
for col in numeric_columns:
    df[col] = df[col].fillna(df[col].median())

# Fill categorical missing values with mode
for col in categorical_columns:
    if df[col].isnull().any():
        df[col] = df[col].fillna(df[col].mode()[0])

# Check missing values after cleaning
print("\nMissing values after cleaning:")
print(df.isnull().sum())

Missing values before cleaning:
user_id      0
```

Before Cleaning:OUTPUT

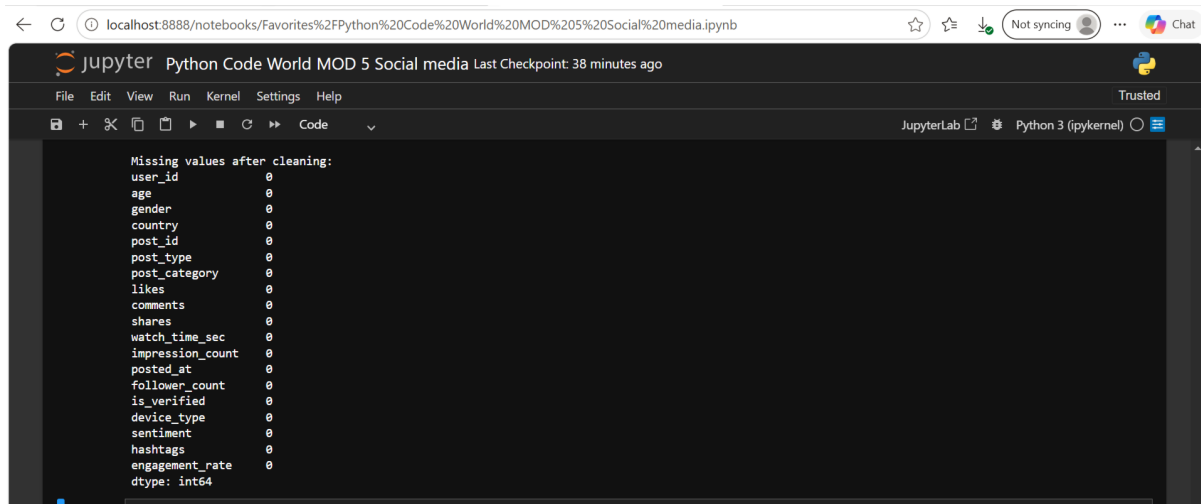
```
localhost:8888/notebooks/Favorites%2FPython%20Code%20World%20MOD%205%20Social%20media.ipynb
Jupyter Python Code World MOD 5 Social media Last Checkpoint: 37 minutes ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

# Fill categorical missing values with mode
for col in categorical_columns:
    if df[col].isnull().any():
        df[col] = df[col].fillna(df[col].mode()[0])

# Check missing values after cleaning
print("\nMissing values after cleaning:")
print(df.isnull().sum())

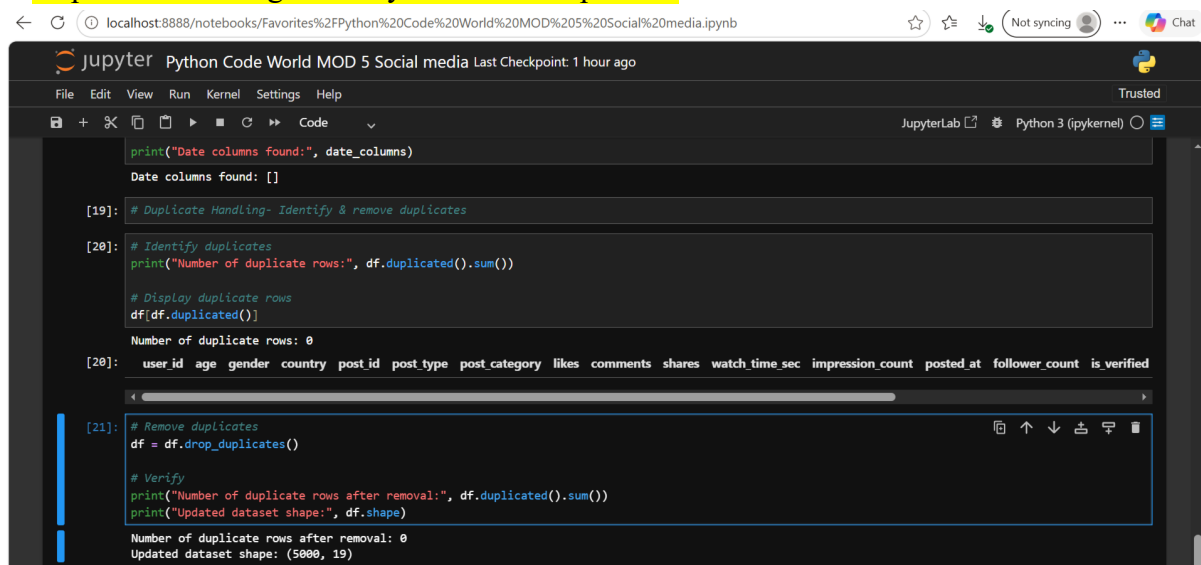
Missing values before cleaning:
user_id      0
age          150
gender       150
country      0
post_id      0
post_type    0
post_category 0
likes        150
comments     150
shares       150
watch_time_sec 0
impression_count 0
posted_at    0
follower_count 0
is_verified  0
device_type  0
sentiment    150
hashtags     0
engagement_rate 0
```

After cleaning: OUTPUT



```
Missing values after cleaning:
user_id      0
age          0
gender       0
country      0
post_id      0
post_type    0
post_category 0
likes        0
comments     0
shares       0
watch_time_sec 0
impression_count 0
posted_at    0
follower_count 0
is_verified  0
device_type  0
sentiment    0
hashtags     0
engagement_rate 0
dtype: int64
```

- **Duplicate Handling- Identify & remove duplicates**



```
print("Date columns found:", date_columns)
Date columns found: []

[19]: # Duplicate Handling- Identify & remove duplicates

[20]: # Identify duplicates
print("Number of duplicate rows:", df.duplicated().sum())

# Display duplicate rows
df[df.duplicated()]

Number of duplicate rows: 0

[20]: user_id age gender country post_id post_type post_category likes comments shares watch_time_sec impression_count posted_at follower_count is_verified

[21]: # Remove duplicates
df = df.drop_duplicates()

# Verify
print("Number of duplicate rows after removal:", df.duplicated().sum())
print("Updated dataset shape:", df.shape)

Number of duplicate rows after removal: 0
Updated dataset shape: (5000, 19)
```

- **Data Formatting**

- Fix incorrect data types
- Standardize categories (e.g., gender labels)
- Correct unrealistic values in likes, comments, shares

```
localhost:8888/notebooks/Favorites%2FPython%20Code%20World%20MOD%205%20Social%20media.ipynb
Jupyter Python Code World MOD 5 Social media Last Checkpoint: 1 hour ago
File Edit View Run Kernel Settings Help Trusted
engagement_rate float64
dtype: object

[24]: df["likes"] = pd.to_numeric(df["likes"], errors="coerce")
      df["comments"] = pd.to_numeric(df["comments"], errors="coerce")
      df["shares"] = pd.to_numeric(df["shares"], errors="coerce")

[25]: # Standardize categories
      df["gender"].unique()

[25]: <ArrowStringArray>
      ['Female', 'Male', 'Other']
      Length: 3, dtype: str

[27]: df["gender"] = df["gender"].astype(str).str.strip().str.lower()

      df["gender"] = df["gender"].replace({
          "m": "Male",
          "male": "Male",
          "f": "Female",
          "female": "Female"
      })
      print(df["gender"].unique())

      <ArrowStringArray>
      ['Female', 'Male', 'other']
      Length: 3, dtype: str
```

• Feature Cleaning

■ Extract hashtag count

```
localhost:8888/notebooks/Favorites%2FPython%20Code%20World%20MOD%205%20Social%20media.ipynb
Jupyter Python Code World MOD 5 Social media Last Checkpoint: 1 hour ago
File Edit View Run Kernel Settings Help Trusted
[33]: print(df["hashtags"].head())

      0 #foodie #travel #love
      1 #fitness
      2 #foodie
      3 #music #foodie #fun
      4 #travel
      Name: hashtags, dtype: str

[34]: import re

      df["hashtag_count"] = df["hashtags"].apply(
          lambda x: len(re.findall(r"#\w+", str(x)))
      )

      df[["hashtags", "hashtag_count"]].head()

[34]:      hashtags  hashtag_count
0 #foodie #travel #love      3
1 #fitness      1
2 #foodie      1
3 #music #foodie #fun      3
4 #travel      1
```

■ Clean sentiment labels

```
localhost:8888/notebooks/Favorites%2FPython%20Code%20World%20MOD%205%20Social%20media.ipynb
Jupyter Python Code World MOD 5 Social media Last Checkpoint: 1 hour ago
File Edit View Run Kernel Settings Help Trusted
1 #fitness 1
2 #foodie 1
3 #music #foodie #fun 3
4 #travel 1

[35]: # 2. Clean Sentiment Labels

[36]: print(df["sentiment"].unique())

      <ArrowStringArray>
      ['negative', 'positive', 'neutral']
      Length: 3, dtype: str

[ ]:
```

❖ Task 3 — Data Exploration using Pandas

Perform the following:

- View dataset structure using head(), tail(), shape, and columns.
- Check data types and info with info() and dtypes.

- Generate summary statistics using describe().

```
[42]: # First 5 rows:
df.head()

# Last 5 rows:
df.tail()

# Number of rows and columns:
df.shape

# Column names:
df.columns

# 2. Check data types and information
df.info()

<class 'pandas.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 20 columns):
# Column      Non-Null Count  Dtype
---
0  user_id      5000 non-null   int64
1  age          5000 non-null   float64
2  gender       5000 non-null   str
3  country      5000 non-null   str
4  post_id      5000 non-null   int64
```

- Analyze categorical distributions using value_counts(), unique(), and nunique().
- Create a correlation matrix for numeric fields.

```
[44]: # 3. Generate summary statistics
# For numeric columns:
df.describe()

# For both numeric and categorical columns:
df.describe(include="all")

count    user_id    age  gender  country    post_id  post_type  post_category    likes  comments  shares  watch_time_sec  impression_count
unique      NaN      NaN      3      10      NaN      4          8      NaN      NaN      NaN      NaN      NaN
top         NaN      NaN    Male    India      NaN      reel      fitness      NaN      NaN      NaN      NaN      NaN
freq         NaN      NaN    1849    535      NaN      1283      675      NaN      NaN      NaN      NaN      NaN
mean    54561.890800    38.440400      NaN      NaN    548042.909000      NaN      NaN    10106.997400    1502.039800    1002.9106    4014.503200    50013.732800
std    26090.370121    14.687151      NaN      NaN    260646.957267      NaN      NaN    5702.293017    856.393312    570.8552    2308.096459    28844.939104
min    10055.000000    13.000000      NaN      NaN    100068.000000      NaN      NaN    10.000000    0.000000    0.0000    0.000000    105.000000
25%    32309.500000    26.000000      NaN      NaN    322543.500000      NaN      NaN    5235.000000    792.000000    511.0000    2017.750000    24988.250000
50%    54374.500000    38.000000      NaN      NaN    548077.500000      NaN      NaN    10105.500000    1497.000000    1012.0000    4034.500000    49934.500000
75%    77180.500000    51.000000      NaN      NaN    771574.500000      NaN      NaN    14959.000000    2235.250000    1483.0000    6020.250000    74662.250000
```

- Use groupby() to summarize metrics (e.g., avg likes by post type, impressions by country).

```
[46]: # 4. Analyze categorical distributions
# Check unique values
df["post_type"].unique()

# Count unique values
df["post_type"].nunique()

# Frequency distribution
df["post_type"].value_counts()

df["sentiment"].value_counts()

# And country:
df["country"].value_counts()

country
India      535
Canada     513
Brazil     504
UAE        503
USA        501
France     496
UK          493
Australia  493
Germany    490
Japan      472
Name: count, dtype: int64
```

```
[47]: # 5. Create a correlation matrix

# Select the numeric columns:
numeric_df = df.select_dtypes(include="number")

# Create the correlation matrix:
correlation_matrix = numeric_df.corr()

correlation_matrix
```

	user_id	age	post_id	likes	comments	shares	watch_time_sec	impression_count	follower_count	engagement_rate	hashtag_count
user_id	1.000000	-0.006688	0.020051	0.025811	-0.033395	0.013763	-0.016847	0.015326	0.010124	-0.004282	-0.013692
age	-0.006688	1.000000	-0.013153	-0.036322	-0.007284	0.013871	0.005542	0.013322	-0.024894	0.008039	0.007173
post_id	0.020051	-0.013153	1.000000	0.014526	-0.010540	0.001846	0.018374	-0.007709	-0.002844	0.010139	0.007344
likes	0.025811	-0.036322	0.014526	1.000000	-0.018421	0.004712	0.008710	0.007952	-0.022982	0.093520	-0.002190
comments	-0.033395	-0.007284	-0.010540	-0.018421	1.000000	0.006142	-0.016351	-0.009395	-0.011733	0.000051	-0.015230
shares	0.013763	0.013871	0.001846	0.004712	0.006142	1.000000	0.014658	-0.005204	-0.010783	0.021724	0.013379
watch_time_sec	-0.016847	0.005542	0.018374	0.008710	-0.016351	0.014658	1.000000	-0.004335	0.002761	-0.001148	-0.023301
impression_count	0.015326	0.013322	-0.007709	0.007952	-0.009395	-0.005204	-0.004335	1.000000	-0.015513	-0.232226	-0.002714
follower_count	0.010124	-0.024894	-0.002844	-0.022982	-0.011733	-0.010783	0.002761	-0.015513	1.000000	0.002292	0.008802

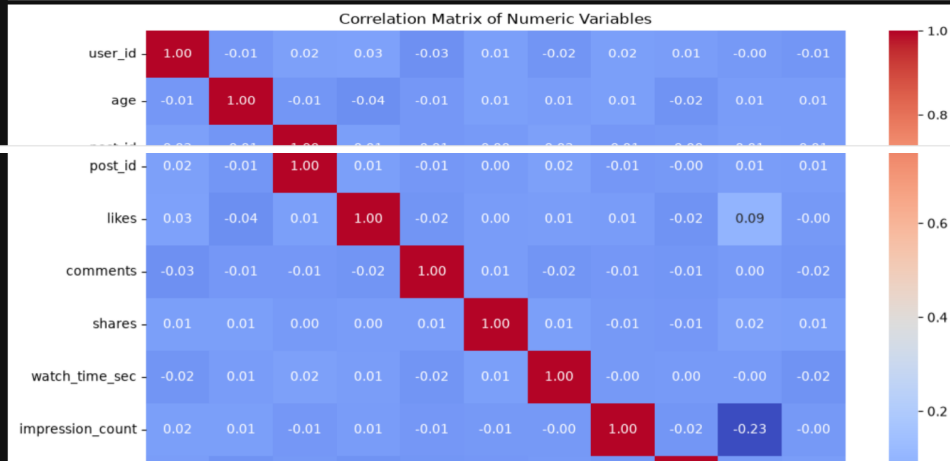
```
[48]: # For a visual correlation matrix, use Seaborn:

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 8))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Matrix of Numeric Variables")
plt.show()
```



localhost:8888/notebooks/Favorites%20Python%20Code%20World%20MOD%205%20Social%20media.ipynb

Jupyter Python Code World MOD 5 Social media Last Checkpoint: 2 hours ago

```
print(avg_likes_by_post_type)

# Average comments by post type
df.groupby("post_type")["comments"].mean()

# Average shares by post type
df.groupby("post_type")["shares"].mean()

# Average impressions by post category
df.groupby("post_category")["impression_count"].mean()

# Average engagement rate by post type
df.groupby("post_type")["engagement_rate"].mean()
```

```
post_type
video    10188.600000
image    10104.865277
text     10100.148193
reel     10037.892416
Name: likes, dtype: float64
```

```
[49]: post_type
image    0.895646
reel     0.783047
text     1.064549
video    1.333955
```

Final OUTPUT:

```
[50]: # 7. Multiple metrics using groupby()

df.groupby("post_type").agg({
    "likes": "mean",
    "comments": "mean",
    "shares": "mean",
    "impression_count": "mean",
    "engagement_rate": "mean"
})
```

```
[50]:
```

	likes	comments	shares	impression_count	engagement_rate
post_type					
image	10104.865277	1525.235766	1019.289495	49727.421812	0.895646
reel	10037.802416	1504.187062	982.042089	50462.598597	0.783047
text	10100.148193	1499.106024	1013.989558	49987.218474	1.064549
video	10188.600000	1479.160000	996.834286	49862.014694	1.122365

❖ Task 4 — Data Wrangling

- Use merge, concat, or join if combining DataFrames.
- Create new fields such as engagement_score, log-transformed metrics (optional), and hashtag count.
- Perform groupby summaries by post_type, country, and sentiment.

Final OUTPUT:

```
display(sentiment_summary)
```

Summary by Post Type:

	likes	comments	shares	impression_count	engagement_score	engagement_rate
post_type						
image	10104.87	1525.24	1019.29	49727.42	12649.39	0.90
reel	10037.80	1504.19	982.04	50462.60	12524.03	0.78
text	10100.15	1499.11	1013.99	49987.22	12613.24	1.06
video	10188.60	1479.16	996.83	49862.01	12664.59	1.12

Summary by Country:

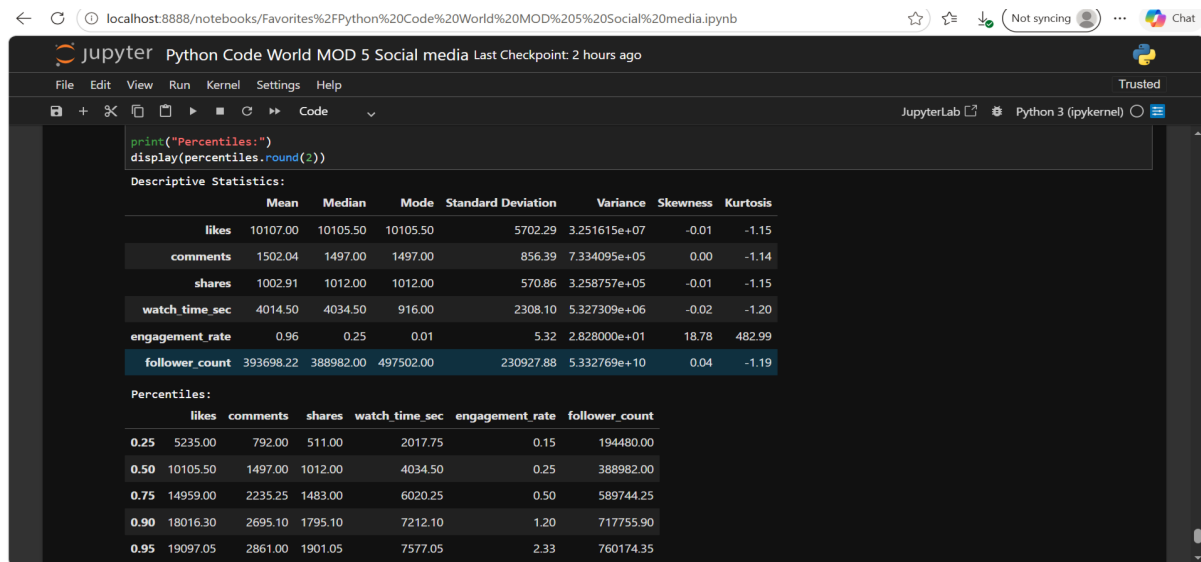
	likes	comments	shares	impression_count	engagement_score	engagement_rate
country						
Australia	10294.12	1485.80	1029.69	48346.38	12809.62	1.32
Brazil	9886.69	1490.43	1008.86	49193.17	12385.98	1.54
Canada	10063.07	1546.58	996.25	48703.15	12605.89	0.92
France	10372.88	1478.70	1051.81	51727.67	12903.38	1.15
Germany	10130.46	1531.83	973.38	48605.41	12635.66	0.76
India	9962.47	1494.92	1035.32	52462.39	12492.70	0.66

❖ Task 5 — Statistical Analysis

Compute descriptive stats for “likes, comments, shares, watch_time, engagement_rate, followers” columns:

- Mean, median, mode
- Standard deviation, variance
- Percentiles
- (Optional) Skewness and kurtosis

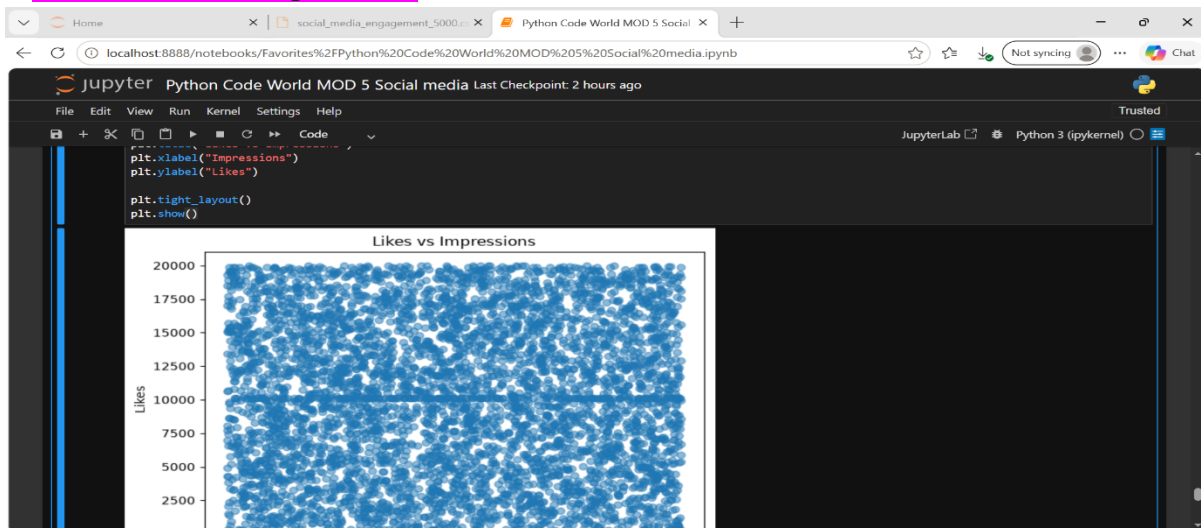
Final OUTPUT:



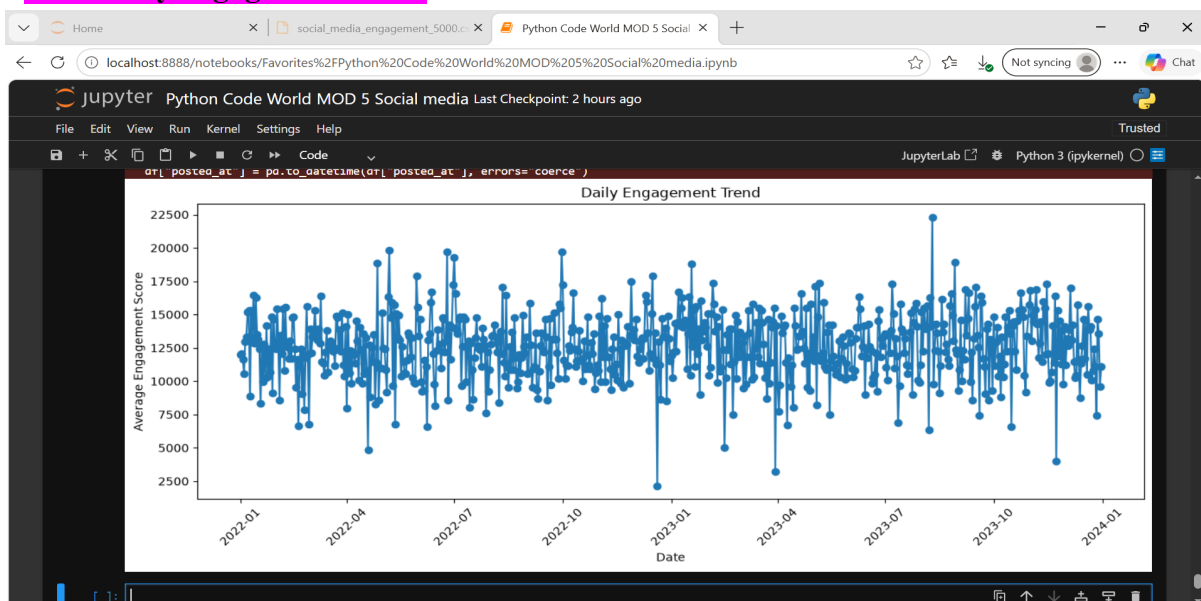
❖ Task 6 — Data Visualization (Min. 8 Plots Required)

● Matplotlib

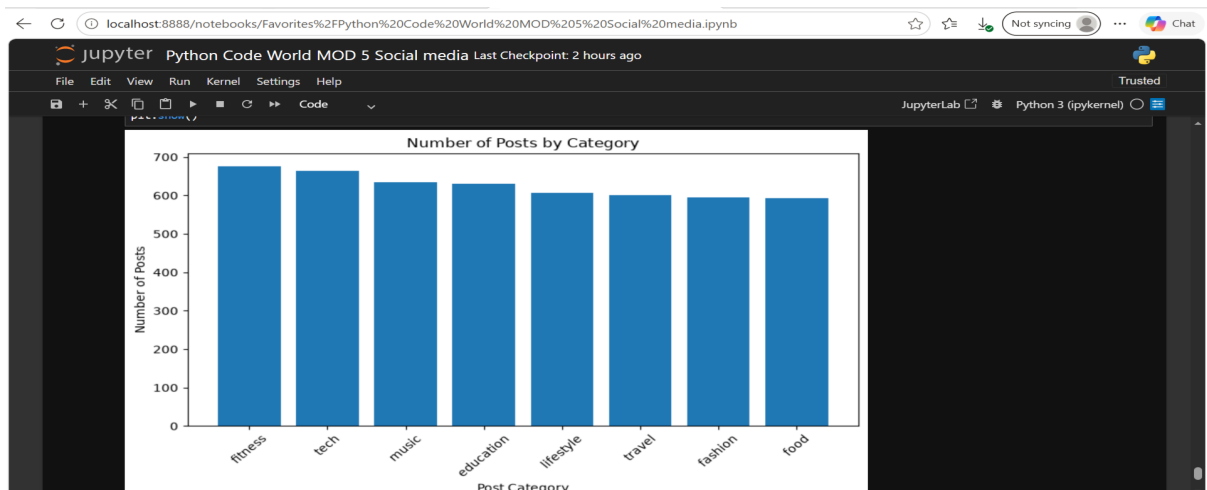
■ Scatter: likes vs impressions



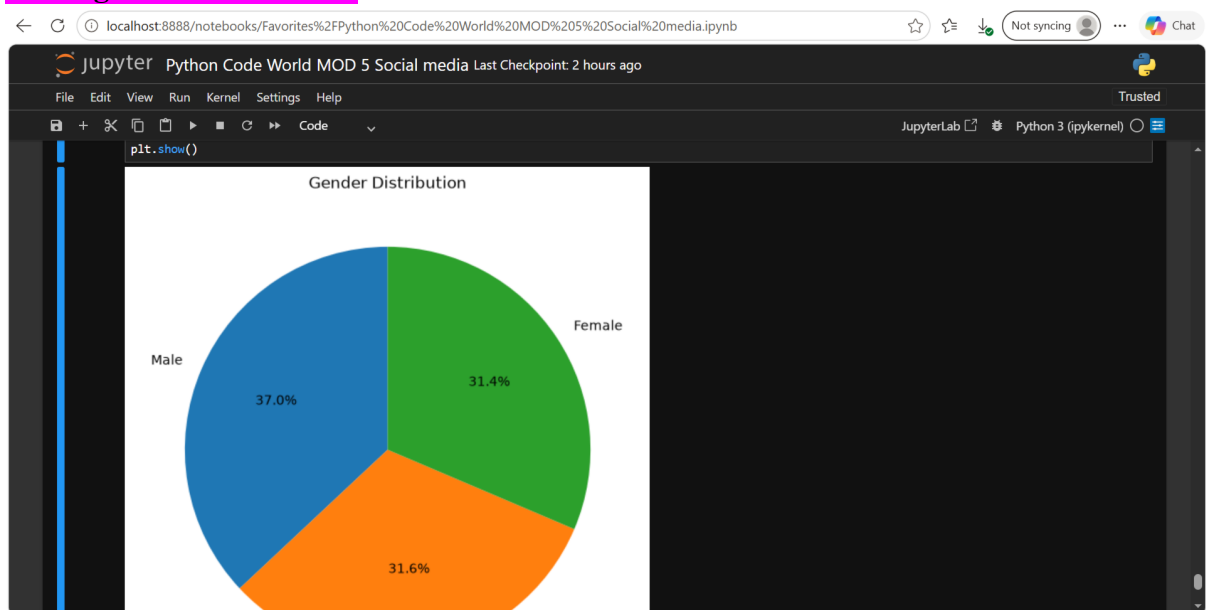
■ Line: daily engagement trend



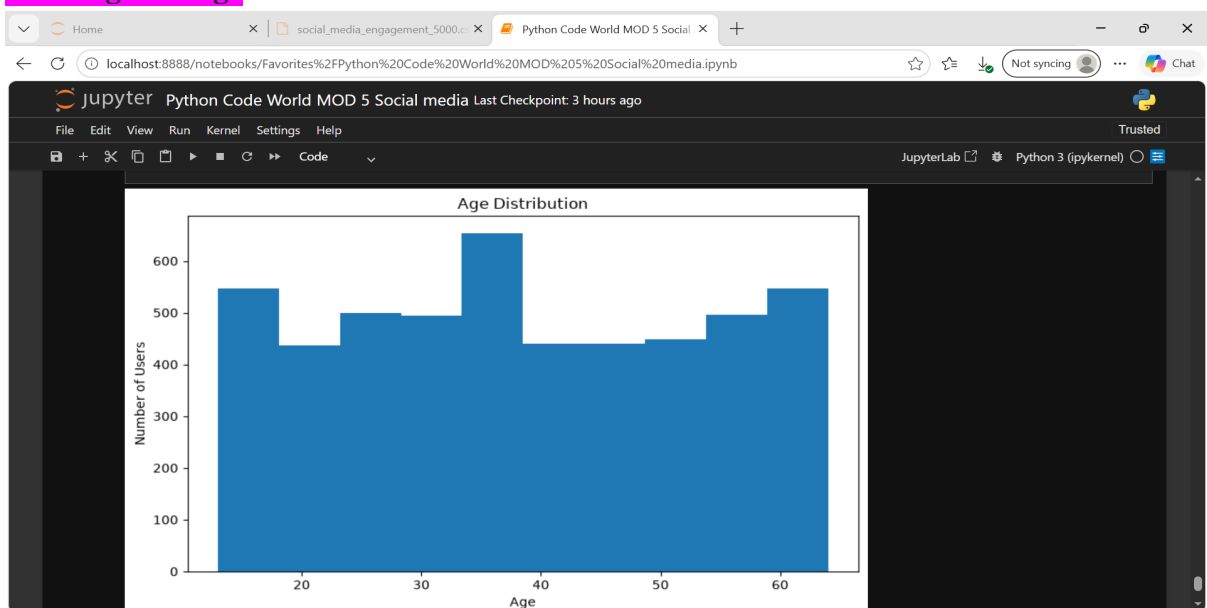
■ Bar: posts by category



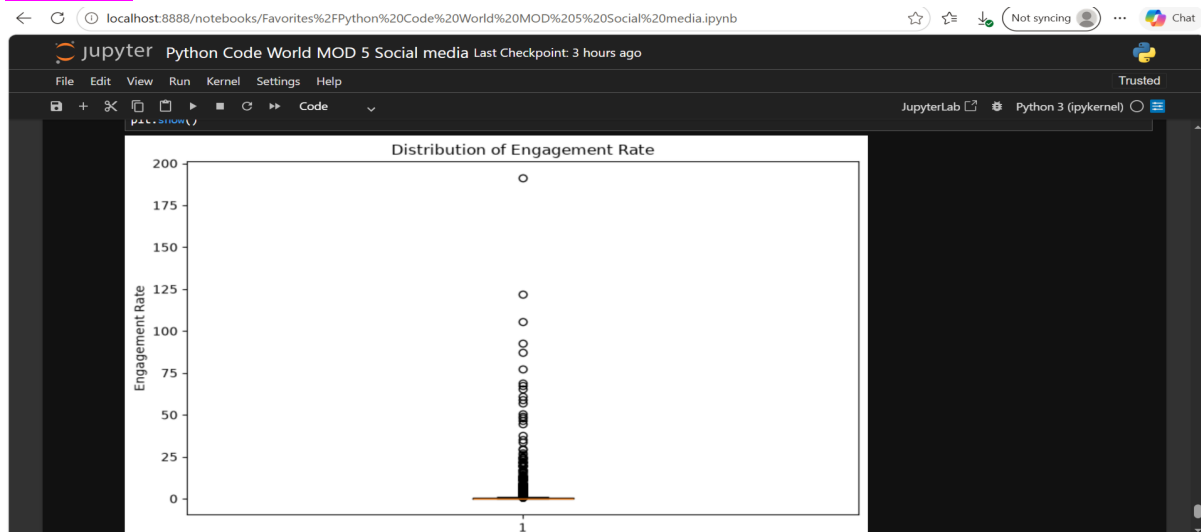
■ Pie: gender distribution



■ Histogram: age



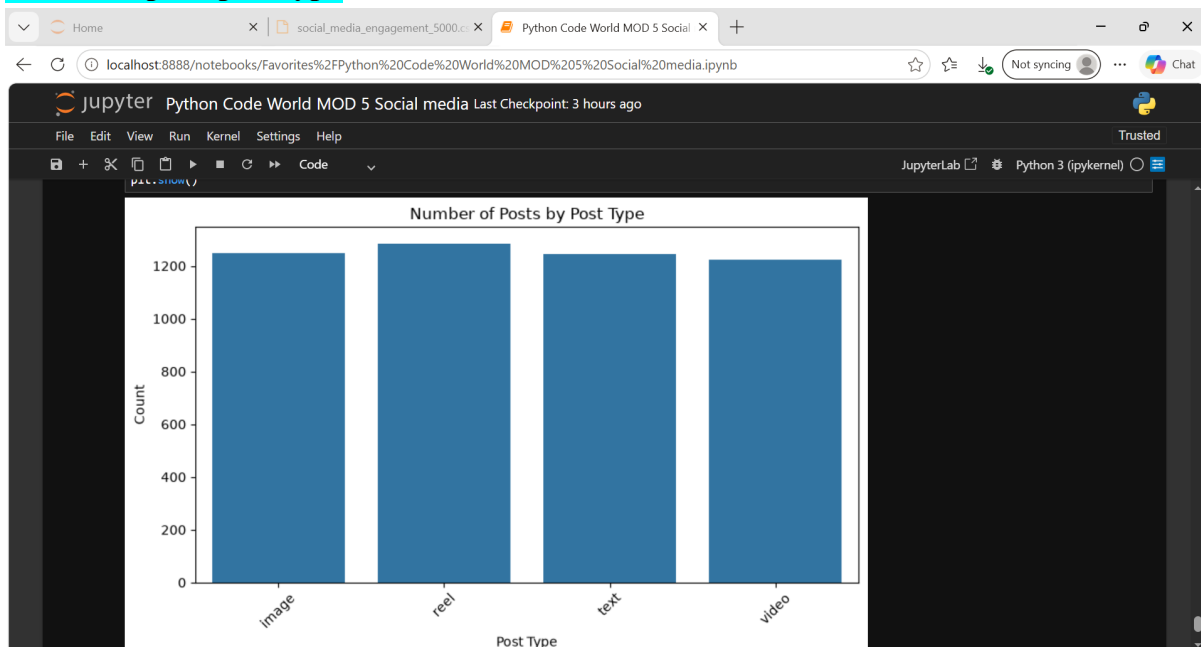
BoxPolt:



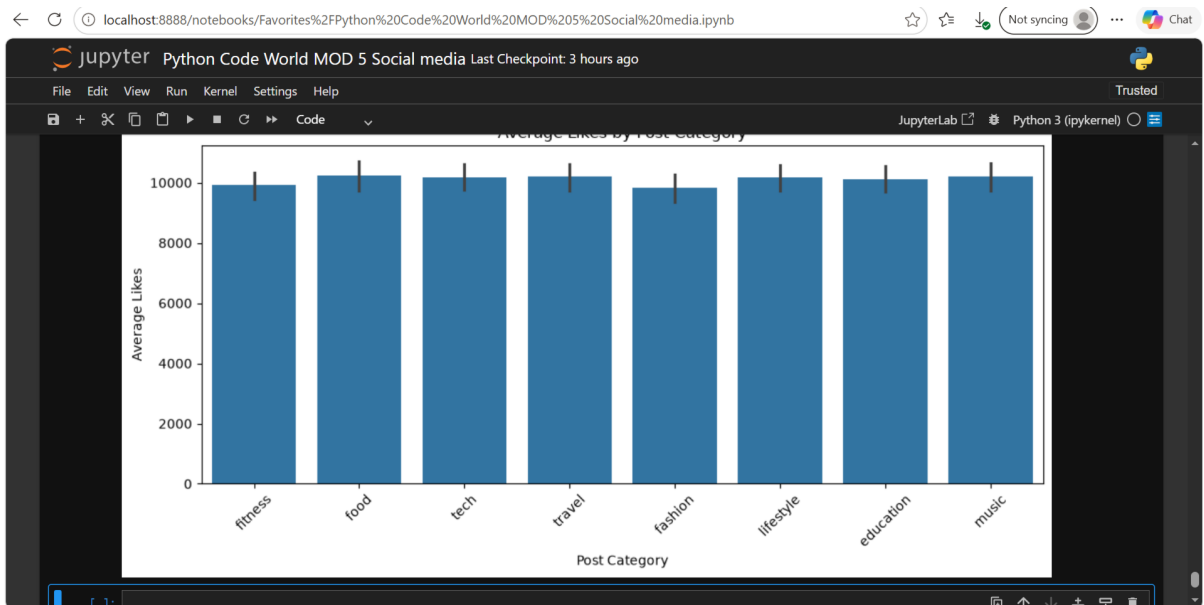
#	Visualization	Variables
1	Scatter Plot	Likes vs Impressions
2	Line Plot	Daily Engagement Trend
3	Bar Chart	Posts by Category
4	Pie Chart	Gender Distribution
5	Histogram	Age
6	Box Plot	Engagement Rate

• Seaborn

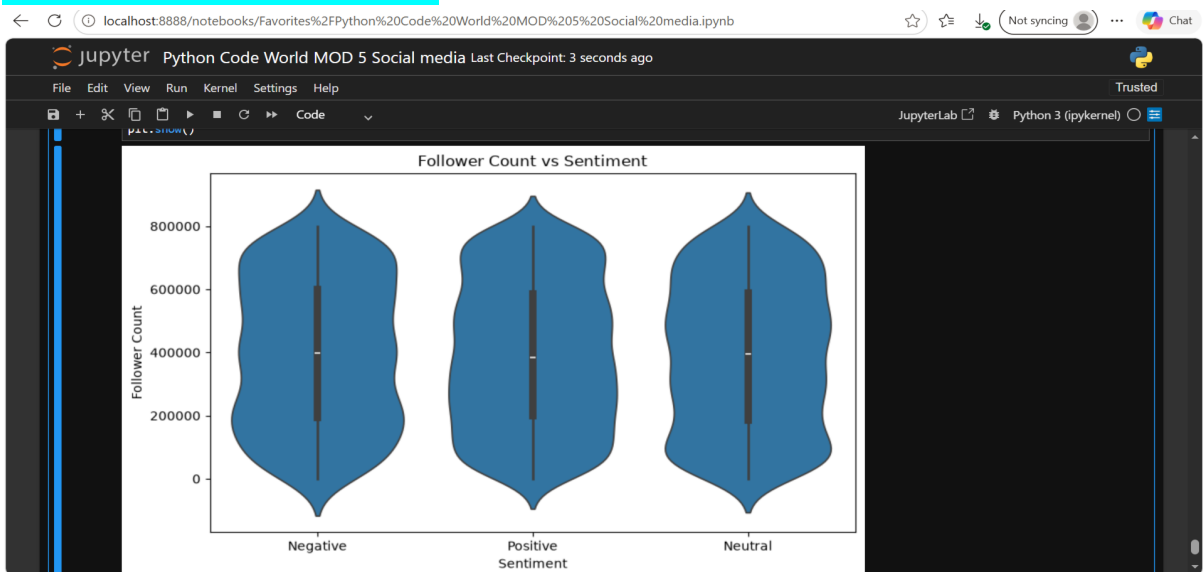
➤ Count plot: post type



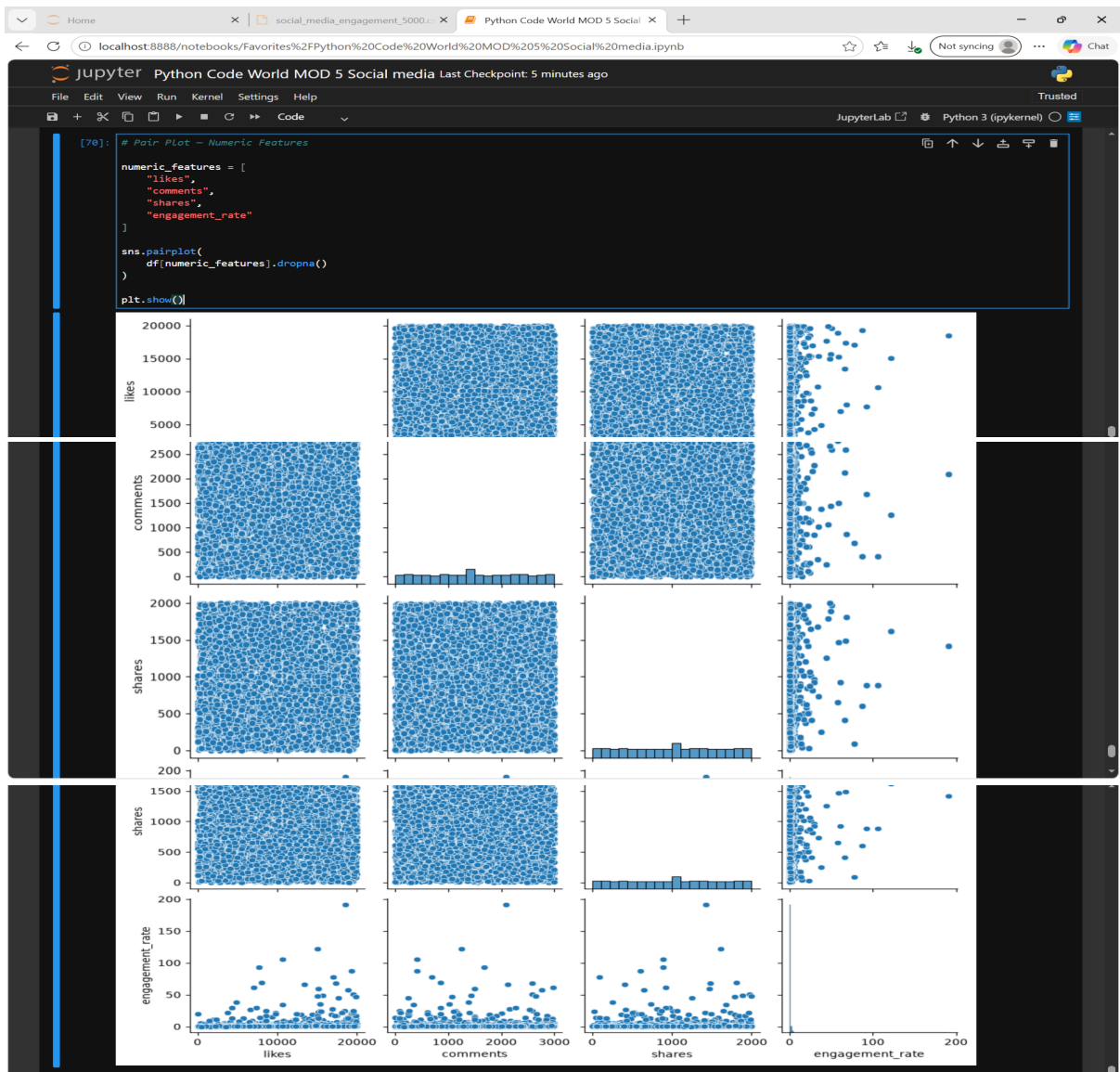
➤ Bar graph: avg likes by category:



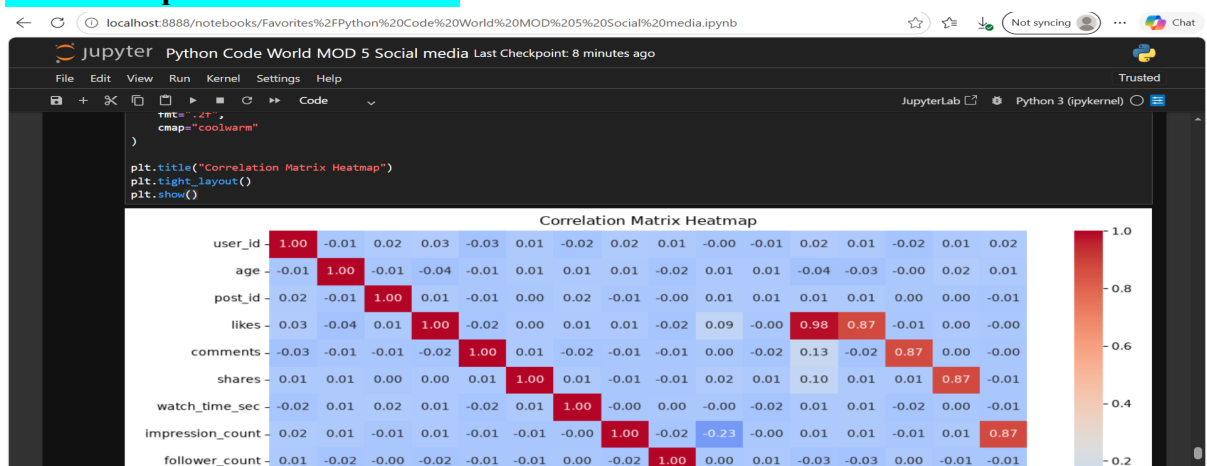
➤ Violin: followers vs sentiment

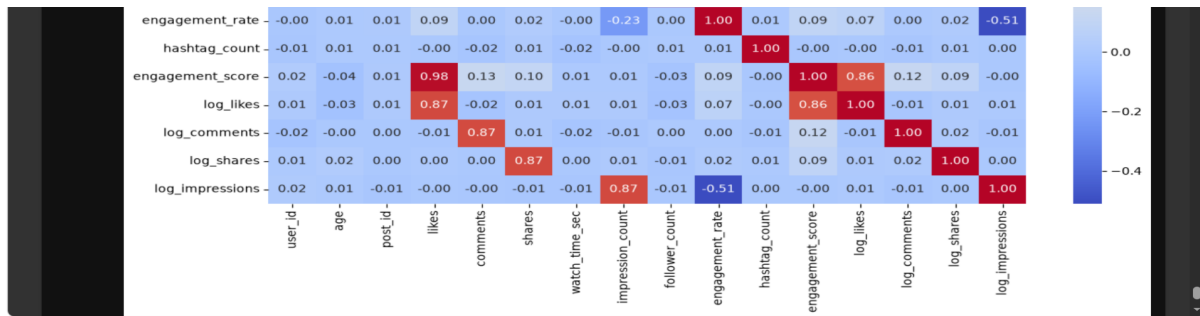


➤ Pair plot: numeric features

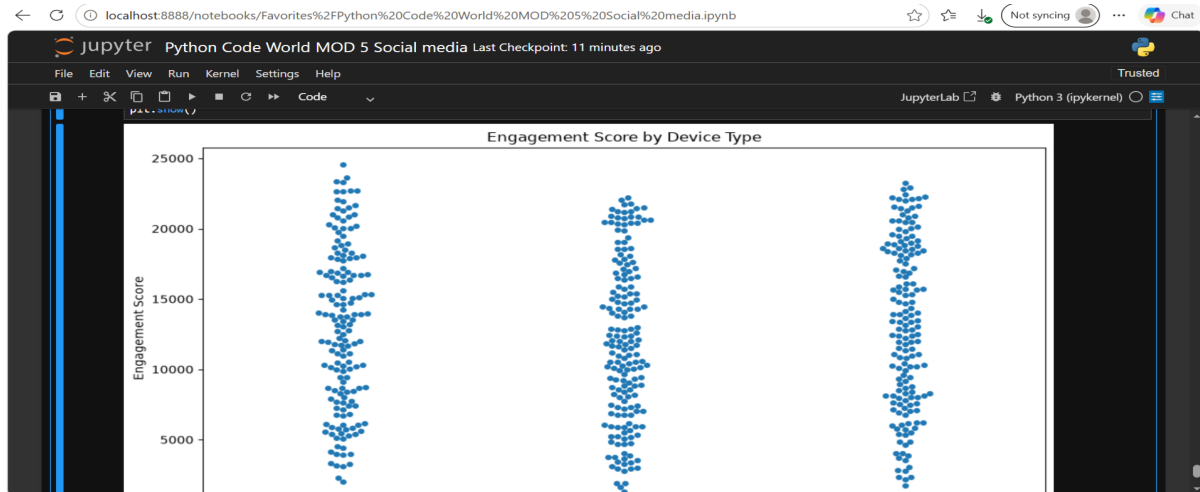


➤ Heatmap: correlation matrix:



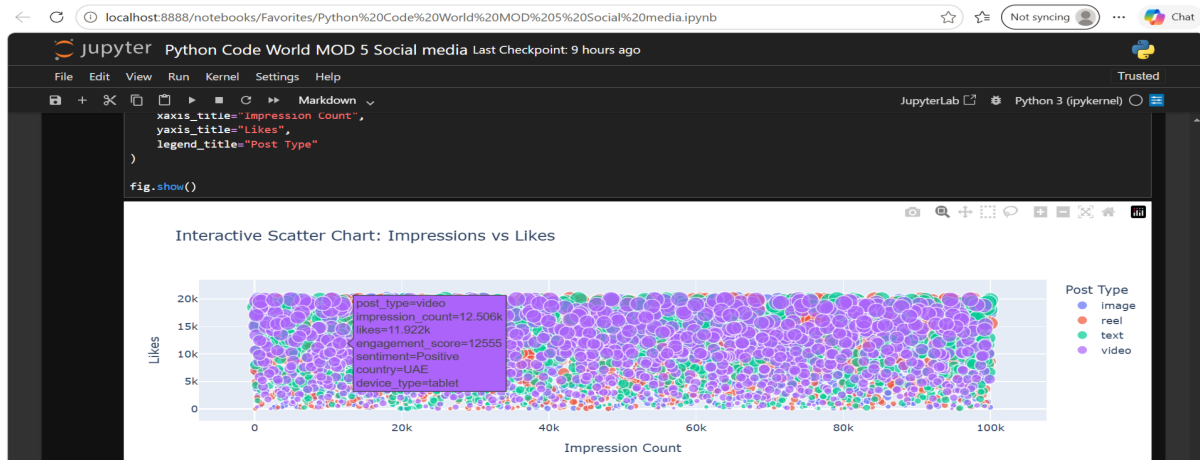


➤ Swarm plot: engagement vs device:



● Plotly (Interactive)

✓ Interactive line chart/bar chart/bubble/scatter char



❖ Final Insights should include the following analysis:

● Content Performance

✓ Which post types have the highest engagement?

```

# Likes + Comments + Shares

df["engagement"] = df["likes"] + df["comments"] + df["shares"]

post_type_engagement = (
    df.groupby("post_type")["engagement"]
      .mean()
      .sort_values(ascending=False)
)

print("Average Engagement by Post Type:")
print(post_type_engagement)

Average Engagement by Post Type:
post_type
video      12658.675844
image      12627.062882
reel       12560.733850
text       12529.878199
Name: engagement, dtype: float64

[10]: print("\nHighest Engagement Post Type:")
      print(post_type_engagement.idxmax())

Highest Engagement Post Type:
video
```

✓ Best-performing content category?

```

# column has post_category, not category.

category_engagement = (
    df.groupby("post_category")["engagement"]
      .mean()
      .sort_values(ascending=False)
)

print("Average Engagement by Content Category:")
print(category_engagement)

Average Engagement by Content Category:
post_category
music      12896.741710
food       12656.661710
tech       12649.633117
lifestyle  12616.940109
travel     12595.420857
education  12594.467826
fitness    12483.886614
fashion    12249.938889
Name: engagement, dtype: float64

[12]: print("\nBest-Performing Content Category:")
      print(category_engagement.idxmax())

Best-Performing Content Category:
music
```

✓ Which countries have the highest average engagement rate?

```

music

[13]: # 3 Which countries have the highest average engagement rate?
      # dataset already has an engagement_rate column, so we can use it directly.

      country_engagement = (
          df.groupby("country")["engagement_rate"]
            .mean()
            .sort_values(ascending=False)
      )

      print("Average Engagement Rate by Country:")
      print(country_engagement)

Average Engagement Rate by Country:
country
Brazil      1.540704
Australia   1.324339
France      1.146402
UAE         1.112352
Canada      0.916659
UK          0.850966
Japan       0.769623
Germany     0.759000
India       0.655005
USA         0.576580
Name: engagement_rate, dtype: float64
```

• User Trends

1. How Age Affects Engagement

A scatter plot is useful here because age is numeric and engagement is numeric.

✓ First calculate average engagement by age:

```
localhost:8888/notebooks/Favorites/Python%20Code%20World%20MOD%205%20Social%20media.ipynb
Jupyter Python Code World MOD 5 Social media Last Checkpoint: 9 hours ago
File Edit View Run Kernel Settings Help Trusted
+ % Copy Paste Run Cell Run All Code
Name: engagement_rate, dtype: float64

[16]: # User Trends

[17]: # 1. How Age Affects Engagement

# A scatter plot is useful here because age is numeric and engagement is numeric.

# First calculate average engagement by age:

age_engagement = (
    df.groupby("age")["engagement"]
      .mean()
      .sort_index()
)

print("Average Engagement by Age:")
print(age_engagement)

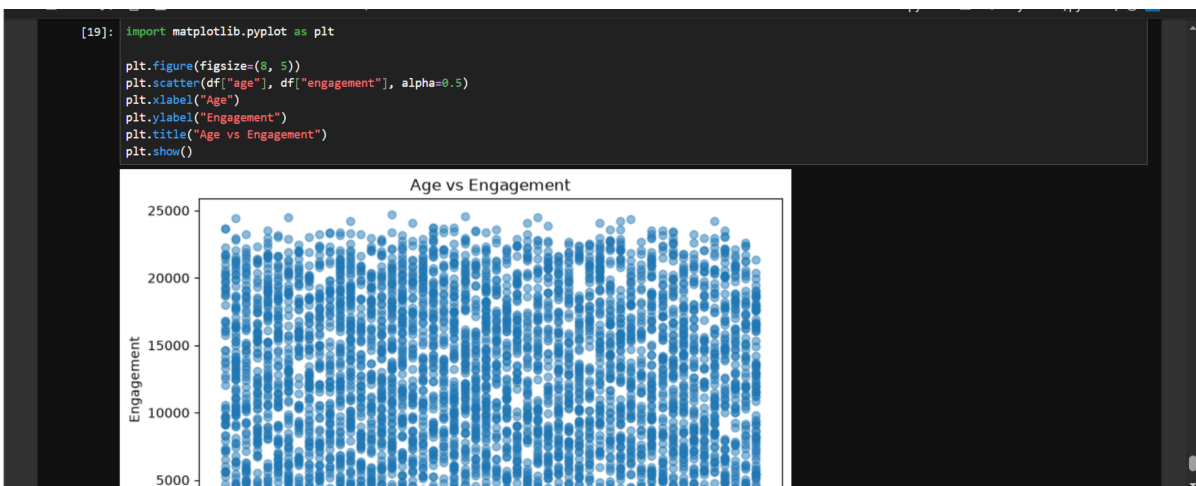
Average Engagement by Age:
age
13.0    12680.452055
14.0    13228.698925
15.0    12900.486486
16.0    12330.134146
17.0    14053.046512
18.0    12477.159091
19.0    11839.535714
20.0    13233.418919
```

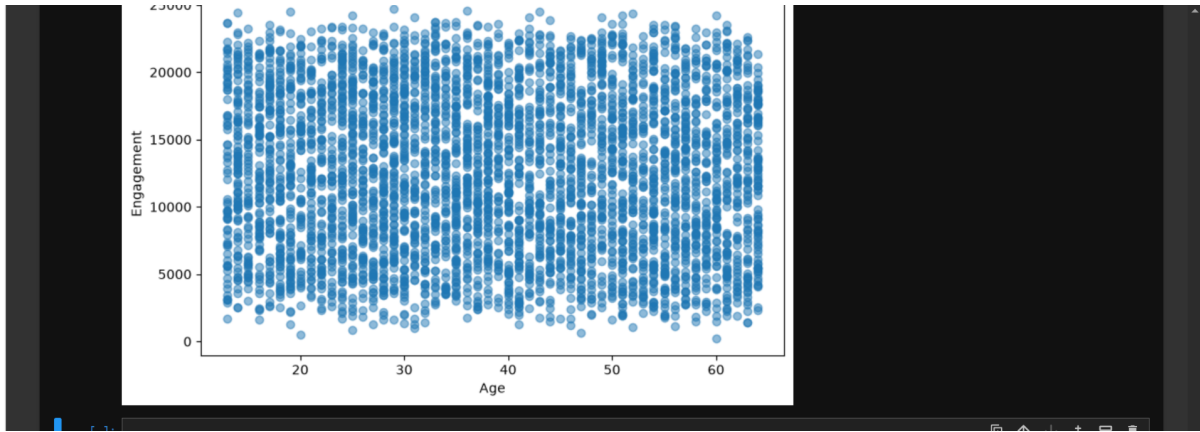
```
52.0    12433.551282
53.0    11341.507246
54.0    12110.096154
55.0    11502.551724
56.0    12779.714286
57.0    12283.626374
58.0    11594.291139
59.0    11972.592105
60.0    12089.641975
61.0    12378.804598
62.0    12710.000000
63.0    11907.000000
64.0    12053.233333
Name: engagement, dtype: float64

[18]: age_correlation = df["age"].corr(df["engagement"])

print("Correlation between Age and Engagement:")
print(age_correlation)

Correlation between Age and Engagement:
-0.03761822515508393
```





2. Performance difference for verified accounts

```
localhost:8888/notebooks/Favorites/Python%20Code%20World%20MOD%205%20Social%20media.ipynb
```

Jupyter Python Code World MOD 5 Social media Last Checkpoint: 10 hours ago

```
File Edit View Run Kernel Settings Help
```

Code

```
[20]: # 2. Performance Difference for Verified Accounts

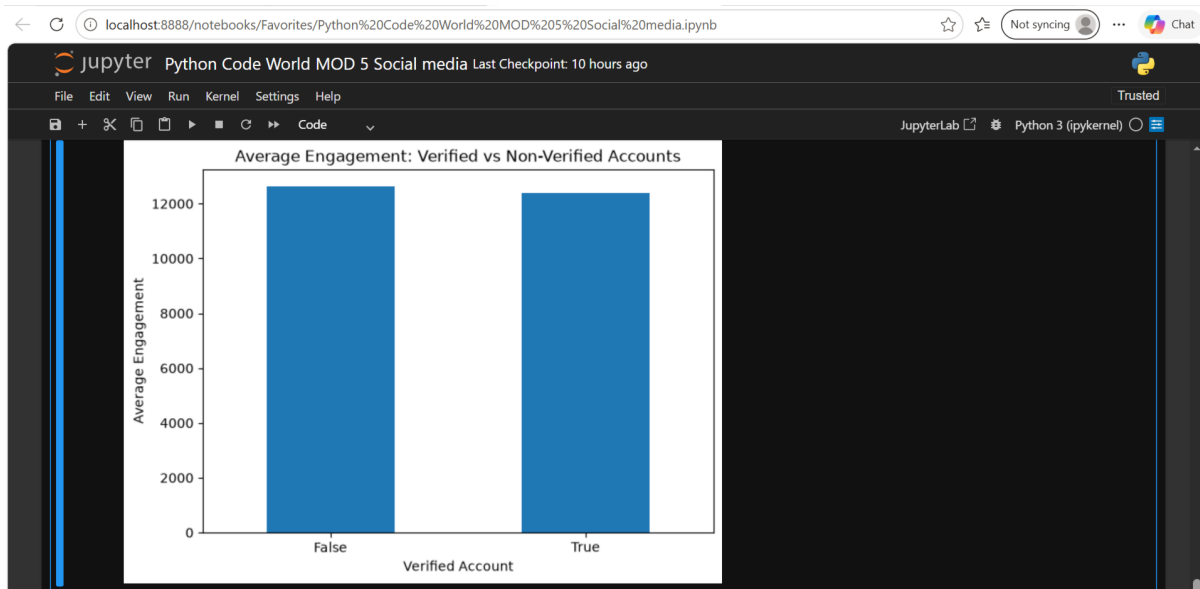
# the account is verified

verified_performance = (
    df.groupby("is_verified")[["engagement", "engagement_rate"]]
    .mean()
)

print("Performance by Verification Status:")
print(verified_performance)

Performance by Verification Status:
engagement  engagement_rate
is_verified
False       12617.413223      0.954744
True        12379.117517      1.054250
```

A bar chart makes the verified vs non-verified comparison easy to understand:



```
[21]: import matplotlib.pyplot as plt

verified_performance["engagement"].plot(
    kind="bar",
    figsize=(7, 5)
)

plt.xlabel("Verified Account")
plt.ylabel("Average Engagement")
plt.title("Average Engagement: Verified vs Non-Verified Accounts")
plt.xticks(rotation=0)
plt.show()
```

• Behavioral Insights

✓ Best time of day for impressions

First, create a time-of-day column from posted_at.

```
[22]: # Behavioral Insights

[24]: # First, create a time-of-day column from posted_at.

# Convert posted_at to datetime
df["posted_at"] = pd.to_datetime(df["posted_at"])

# Extract hour
df["hour"] = df["posted_at"].dt.hour

# Calculate average impressions by hour
hourly_impressions = (
    df.groupby("hour")["impression_count"]
    .mean()
    .sort_values(ascending=False)
)

print("Average Impressions by Hour:")
print(hourly_impressions)

C:\Users\User\AppData\Local\Temp\ipykernel_11988\3063220169.py:4: UserWarning: Parsing dates in %d-%m-%Y format when dayfirst=False (the default) was specified. Pass `dayfirst=True` or specify a format to silence this warning.
df["posted_at"] = pd.to_datetime(df["posted_at"])

Average Impressions by Hour:
hour
0    50013.7328
Name: impression_count, dtype: float64
```

To identify the hour with the highest average impressions:

```
localhost:8888/notebooks/Favorites/Python%20Code%20World%20MOD%205%20Social%20media.ipynb

Jupyter Python Code World MOD 5 Social media Last Checkpoint: 10 hours ago

File Edit View Run Kernel Settings Help Trusted

[25]: # To identify the hour with the highest average impressions:

best_hour = hourly_impressions.idxmax()
best_impressions = hourly_impressions.max()

print("Best Time of Day for Impressions:")
print(f"{best_hour}:00")
print(f"Average Impressions: {best_impressions:.2f}")

Best Time of Day for Impressions:
0:00
Average Impressions: 50013.73
```

✓ Device type impact on watch time

Now compare watch_time_sec across different devices.

```
Python Code World MOD 5 Social media Last Checkpoint: 10 hours ago

File Edit View Run Kernel Settings Help Trusted

[26]: # 2. Device Type Impact on Watch Time

# compare watch_time_sec across different devices.

device_watch_time = (
    df.groupby("device_type")["watch_time_sec"]
      .mean()
      .sort_values(ascending=False)
)

print("Average Watch Time by Device Type:")
print(device_watch_time)

Average Watch Time by Device Type:
device_type
mobile      4087.830760
tablet      3979.736429
desktop     3974.792521
Name: watch_time_sec, dtype: float64
```

```
Python Code World MOD 5 Social media Last Checkpoint: 10 hours ago

File Edit View Run Kernel Settings Help Trusted

[27]: # Find the device with the highest average watch time:

[28]: best_device = device_watch_time.idxmax()
best_watch_time = device_watch_time.max()

print("\nDevice with Highest Average Watch Time:")
print(best_device)
print(f"Average Watch Time: {best_watch_time:.2f} seconds")

Device with Highest Average Watch Time:
mobile
Average Watch Time: 4087.83 seconds

[29]: # Visualization

# A bar chart will make the device comparison clear:

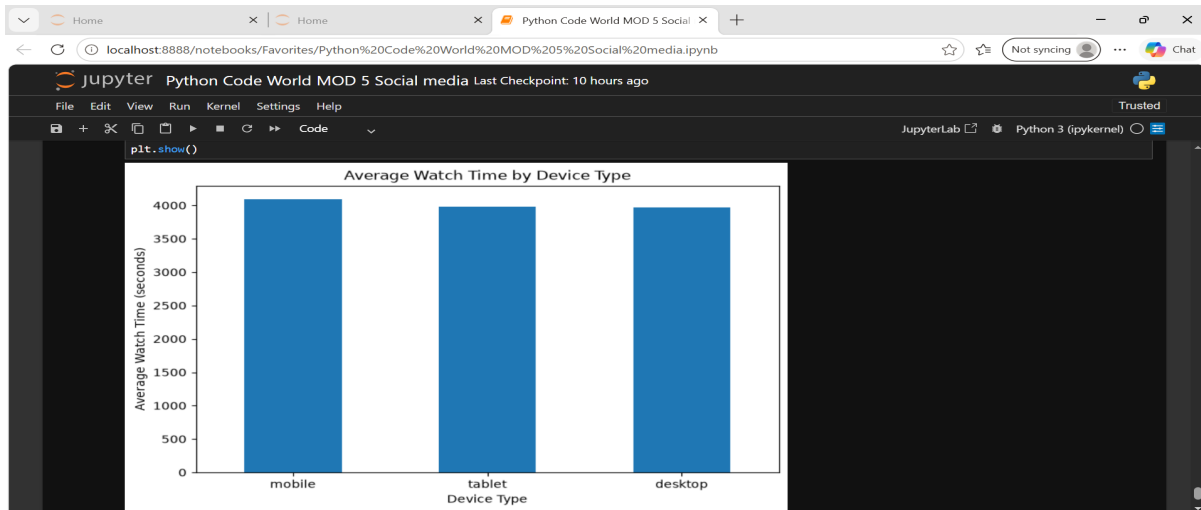
import matplotlib.pyplot as plt

device_watch_time.plot(
    kind="bar",
    figsize=(8, 5)
)

plt.xlabel("Device Type")
plt.ylabel("Average Watch Time (seconds)")
```

Visualization

A bar chart will make the device comparison clear:



• Sentiment Analysis

1. Which sentiment performs best?

Since we already created the engagement column:

```
[32]: # 1. Which Sentiment Performs Best

# Since we already created the engagement column:
sentiment_performance = (
    df.groupby("sentiment")[["engagement", "engagement_rate", "impression_count"]]
      .mean()
      .sort_values("engagement", ascending=False)
)

print("Sentiment Performance:")
print(sentiment_performance)

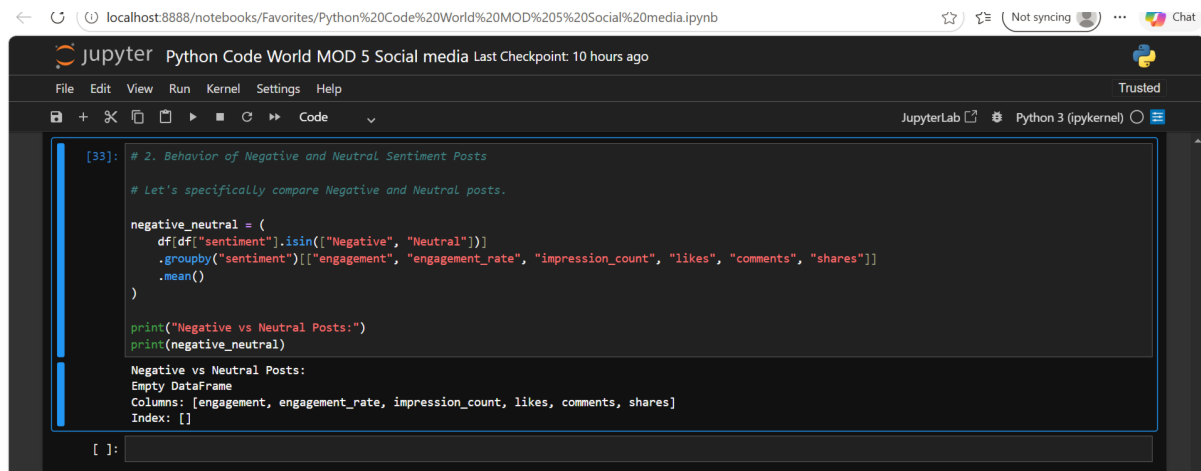
# To identify the sentiment with the highest average engagement:
best_sentiment = sentiment_performance["engagement"].idxmax()
best_engagement = sentiment_performance["engagement"].max()

print("\nBest-Performing Sentiment:")
print(best_sentiment)
print(f"Average Engagement: {best_engagement:.2f}")

Sentiment Performance:
           engagement  engagement_rate  impression_count
sentiment
negative    12745.962670             1.038486         50191.960417
positive    12593.029963             0.954800         49986.155734
neutral     12459.355160             0.991170         49515.971185

Best-Performing Sentiment:
negative
Average Engagement: 12745.96
```

2 Behavior of negative/neutral sentiment posts:



```
[33]: # 2. Behavior of Negative and Neutral Sentiment Posts
# Let's specifically compare Negative and Neutral posts.

negative_neutral = (
    df[df["sentiment"].isin(["Negative", "Neutral"])]
    .groupby("sentiment")[["engagement", "engagement_rate", "impression_count", "likes", "comments", "shares"]]
    .mean()
)

print("Negative vs Neutral Posts:")
print(negative_neutral)

Negative vs Neutral Posts:
Empty DataFrame
Columns: [engagement, engagement_rate, impression_count, likes, comments, shares]
Index: []
```

Conclusion

The **Python Social Media Data Analysis & Visualization project** provided practical experience in analyzing and interpreting social media data using Python. The project covered the complete data analysis process, starting from data loading and cleaning to exploratory analysis, visualization, and insight generation.

Using **Pandas and NumPy**, the dataset was prepared and analyzed by examining user information, post performance, engagement, impressions, watch time, device usage, and sentiment. A total engagement metric was also created using likes, comments, and shares to better understand overall user interaction.

The project used **Matplotlib and Seaborn** to create different static visualizations, including scatter plots, line charts, count plots, bar plots, violin plots, pair plots, heatmaps, and swarm plots. **Plotly** was also used to create interactive visualizations that made it easier to explore relationships and patterns within the data.

The analysis helped identify patterns related to **content performance, user age, verified accounts, posting time, device type, country, and sentiment**. These findings demonstrate how visualization and exploratory data analysis can help transform raw social media data into meaningful insights.

Overall, this project strengthened my practical skills in **Python programming, Pandas, data cleaning, exploratory data analysis, statistical visualization, interactive visualization, and data interpretation**. It also provided valuable experience in presenting analytical results in a clear and understandable manner.